

Algoritmi

Riccardo Torre

23 marzo 2026

Indice

1	Strutture dati	1
1.1	Stack minimo/coda minima	1
1.1.1	Modifica dello stack	1
1.1.2	Modifica della coda – metodo 1	2
1.1.3	Modifica della coda – metodo 2	2
1.1.4	Modifica della coda – metodo 3	3

1 Strutture dati

1.1 Stack minimo/coda minima

1.1.1 Modifica dello stack

Per trovare l'elemento più piccolo in uno stack in $O(1)$ mantenendo il comportamento asintotico per aggiunta e rimozione di elementi, si memorizzano delle coppie.

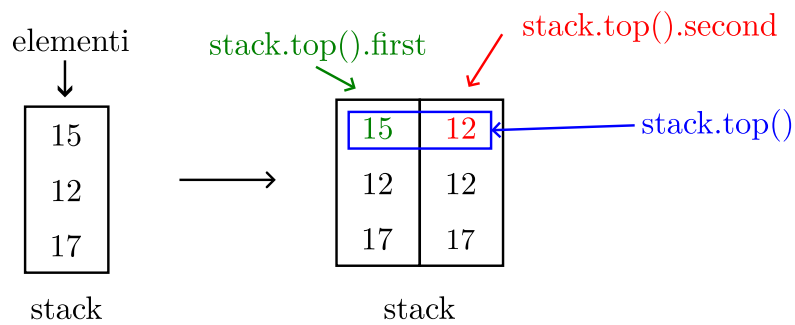


Figura 1: l'elemento minimo viene calcolato ad ogni inserimento e salvato sulla seconda colonna dell'elemento in cima allo stack (in rosso).

Listing 1: aggiungere un elemento.

```
1  int new_min = st.empty() ? new_elem : min(new_elem, st.top().second);
2  st.push({new_elem, new_min});
```

Listing 2: rimuovere un elemento.

```
1 int removed_element = st.top().first;
2 st.pop();
```

Listing 3: trovare il minimo.

```
1 int minimum = st.top().second;
```

Lo stack ha un comportamento LIFO.

1.1.2 Modifica della coda – metodo 1

La coda ha un comportamento FIFO. Non memorizza tutti gli elementi: vengono memorizzati solo quelli necessari per determinare il minimo. La coda viene mantenuta in *ordine non decrescente*¹ (il valore più piccolo sarà memorizzato nella testa). Il minimo effettivo deve sempre essere mantenuto nella coda.

Listing 4: implementazione della coda.

```
1 deque<int> q;
```

Listing 5: trovare il minimo.

```
1 int minimum = q.front();
```

Prima di aggiungere un nuovo elemento nella coda è sufficiente effettuare un “taglio”: vengono rimossi tutti gli elementi finali della coda che sono più grandi del nuovo elemento e successivamente si aggiunge il nuovo elemento alla coda. In questo modo viene mantenuto l’ordine non decrescente e non viene perso l’elemento corrente se in qualsiasi passaggio successivo è l’elemento minimo. Tutti gli elementi che vengono rimossi non potranno mai essere primi.

Listing 6: aggiungere un elemento.

```
1 while (!q.empty() && q.back() > new_element)
2     q.pop_back();
3     q.push_back(new_element);
```

Quando si vuole estrarre un elemento dalla testa, potrebbe non essere presente se è stato rimosso in precedenza con un elemento più piccolo. Pertanto quando si elimina un elemento da una coda bisogna conoscerne il valore. Se la testa della coda ha lo stesso valore, può essere rimossa in sicurezza, altrimenti non viene apportata nessuna modifica.

Listing 7: rimuovere un elemento.

```
1 if (!q.empty() && q.front() == remove_element)
2     q.pop_front();
```

Tutte queste operazioni richiedono $O(1)$.

1.1.3 Modifica della coda – metodo 2

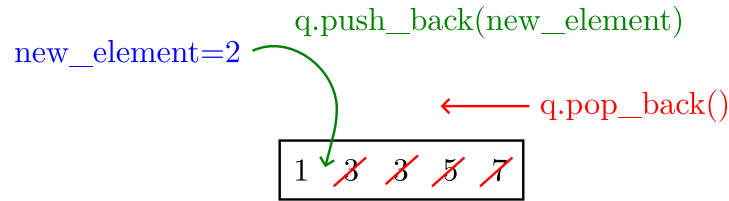
Questa è una modifica del metodo 1. Si memorizza l’indice per ciascun elemento della coda, ricordando anche quanti elementi sono stati già aggiunti e rimossi.

Listing 8: implementazione della coda con contatori di elementi aggiunti e rimossi.

```
1 deque<pair<int, int>> q;
2 int cnt_added = 0;
3 int cnt_removed = 0;
```

¹ cioè $\forall i \in [0, \text{coda.length()} - 1]: \text{coda}[i] \leq \text{coda}[i+1]$

La coda prima dell'inserimento



La coda dopo l'inserimento



Figura 2: effetto dell'inserimento di un nuovo elemento sulla coda.

Listing 9: trovare il minimo.

```
1 int minimum = q.front().first;
```

Listing 10: aggiungere un elemento.

```
1 while (!q.empty() && q.back().first > new_element)
2     q.pop_back();
3     q.push_back({new_element, cnt_added});
4     cnt_added++;
```

La modifica più rilevante è in questa porzione di codice (compararla con quella del metodo 1).

Listing 11: rimuovere un elemento.

```
1 if (!q.empty() && q.front().second == cnt_removed)
2     q.pop_front();
3     cnt_removed++;
```

1.1.4 Modifica della coda – metodo 3

Questo è un altro metodo per trovare il minimo in $O(1)$ modificando una coda. È più complicato da implementare ma permette di memorizzare tutti gli elementi e di rimuovere un elemento dalla parte anteriore senza conoscerne il valore. L'idea è di ridurre il problema al problema degli stack, simulando una coda. Si creano due pile, **s1** e **s2**. Si aggiungono nuovi elementi allo stack **s1**, e si rimuovono da **s2**. Se in qualsiasi momento **s2** è vuoto, si spostano tutti gli elementi da **s1** a **s2** (che inverte l'ordine degli elementi). Infine, per trovare il minimo in una coda è sufficiente trovare il minimo di entrambi gli stack.

Si eseguono tutte le operazioni in $O(1)$ in media (ogni elemento verrà aggiunto una volta allo stack **s1**, una volta trasferito a **s2** e una volta estratto da **s2**).

Listing 12: implementazione.

```
1 stack<pair<int, int>> s1, s2;
```

Listing 13: trovare il minimo.

```
1 if (s1.empty() || s2.empty())
2     minimum = s1.empty() ? s2.top().second : s1.top().second;
3 else
4     minimum = min(s1.top().second, s2.top().second);
```

Listing 14: aggiungi elemento.

```
1  int minimum = s1.empty() ? new_element : min(new_element, s1.top().second);  
2  s1.push({new_element, minimum});
```

Listing 15: rimozione di un elemento.

```
1  if (s2.empty()) {  
2      while (!s1.empty()) {  
3          int element = s1.top().first;  
4          s1.pop();  
5          int minimum = s2.empty() ? element : min(element, s2.top().second);  
6          s2.push({element, minimum});  
7      }  
8  }  
9  int remove_element = s2.top().first;  
10 s2.pop();
```